Projektuppgift

Skapa din egen webbapplikation

DT211G, Frontendutveckling
Francisco Mauricio De Leon Gonzalez

MITTUNIVERSITETET
Institutionen för Informations-system och teknologi

Författare: Francisco Mauricio De Leon Gonzalez, frde2500@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

1 Sammanfattning

Detta projekt syftar till att utveckla en webbapplikation som kombinerar data från flera webbtjänster för att skapa ett användbart verktyg för resenärer. Applikationen, kallad *WeatherTrip*, gör det möjligt för användaren att söka efter en ort och få tillgång till aktuell väderinformation, en väderprognos samt aktivitetsförslag baserade på vädret.

Webbapplikationen använder två JSON-baserade webbtjänster: ett geokodnings-API för att hämta geografiska koordinater för en plats samt ett väder-API för att hämta väderdata baserat på dessa koordinater. Dessa två tjänster integreras till en så kallad mashup.

Projektet har utvecklats med hjälp av JavaScript, SCSS och en automatiserad arbetsprocess baserad på Vite. Versionshantering och publicering har skett med Git och GitHub Actions.

Resultatet är en responsiv webbapplikation med ett applikationsliknande beteende, där majoriteten av innehållet hämtas dynamiskt via API-anrop.

2 Förord

Detta projekt har gett mig en djupare förståelse för hur man planerar, konstruerar och testar en komplett webbapplikation. Jag vill rikta ett tack till kursledningen vid Mittuniversitetet för den tydliga struktur och de praktiska uppgifter som underlättat lärandet inom webbutveckling.

3 Innehållsförteckning

1 Sammanfattning ii
2 Förord iii
3 Innehållsförteckning iv
4 Terminologi v
5 Introduktion 1
5.1 Bakgrund och problemmotivering 1
5.2 Avgränsningar 1
5.3 Syfte och mål 2
5.4 Författarens bidrag 2
6 Teori 3
7 Metod 4
8 Konstruktion 5
8.1 Hantering av användarinput 5
8.2 Integration av API:er 5
8.3 Dynamisk rendering av innehåll 6
8.4 Kartintegration 6
8.5 Lagring av tidigare sökningar 7
8.6 Styling och layout 7
8.7 Animationer 7
8.8 Automatiska arbetsprocess och publicering 7
9 Resultat 8
10 Slutsatser 9

4 Terminologi

Akronymer/Förkortningar

API	Application Programming Interface är ett gränssnitt som möjliggör kommunikation mellan olika system eller applikationer. I detta projekt används API:er för att hämta data från externa webbtjänster.
JSON	JavaScript Object Notation är ett lättviktigt dataformat som används för att strukturera och överföra data mellan system. JSON används i detta projekt för att ta emot data från webbtjänster.
Mashup	En webbapplikation som kombinerar data från flera olika källor för att skapa ny funktionalitet. I detta projekt kombineras data från ett geokodnings-API och ett väder-API.
AJAX	Asynchronous JavaScript and XML är en teknik för att hämta data från en server utan att ladda om sidan. I detta projekt används fetch() för att utföra sådana anrop.
DOM	Document Object Model är en representation av HTML-dokumentet som gör det möjligt att dynamiskt manipulera innehåll med JavaScript.
SCSS	En vidareutveckling av CSS som möjliggör användning av variabler, nesting och modulär kodstruktur.
Responsiv design	En designmetod där webbplatsens layout anpassas efter olika skärmstorlekar och enheter.
Versionshantering (Git)	Ett system för att hantera ändringar i källkod över tid. I projektet används Git tillsammans med GitHub.
Automatiserad arbetsprocess	En process där verktyg används för att automatisera moment som byggning, optimering och publicering av webbapplikationen.

5 Introduktion

5.1 Bakgrund och problemmotivering

I takt med att webben utvecklas ställs allt högre krav på dynamiska och användarvänliga applikationer. Användare förväntar sig att snabbt kunna få tillgång till relevant information utan att behöva navigera mellan flera olika webbplatser.

Detta projekt syftar till att utveckla en webbapplikation som riktar sig till resenärer och som samlar relevant information om en vald plats i ett och samma gränssnitt.

Den centrala problemställningen är att kombinera data från flera olika webbtjänster till en enhetlig lösning. I detta fall handlar det om att hämta geografisk information från ett geokodnings-API och därefter använda denna information för att hämta väderdata från ett väder-API.

Webbapplikationen ska presentera denna information på ett tydligt och användarvänligt sätt, samt komplettera med aktivitetsförslag baserade på väderförhållanden.

Utöver detta ska projektet uppfylla tekniska krav såsom användning av JavaScript för dynamisk rendering, SCSS för styling, samt en automatiserad arbetsprocess för bygg och publicering.

5.2 Avgränsningar

Projektet har avgränsats till att fokusera på grundläggande funktionalitet kopplad till väderdata och platsinformation.

Funktioner som hade kunnat implementeras men som inte ingår i denna lösning är exempelvis:

- integration av ytterligare API:er, såsom sevärdheter eller bilder
- användarkonton och personalisering
- mer avancerade kartfunktioner

Dessa funktioner har uteslutits främst på grund av tidsbegränsningar samt för att hålla projektets omfattning inom kursens ramar.

Fokus har istället legat på att skapa en stabil och fungerande lösning som uppfyller samtliga grundkrav.

5.3 Syfte och mål

Syftet med projektet är att utveckla en webbapplikation riktad mot resenärer, där information från flera olika webbtjänster kombineras till ett enhetligt och användarvänligt gränssnitt. Applikationen ska göra det möjligt för användaren att söka efter en ort och få tillgång till aktuell väderinformation, en korttidsprognos samt aktivitetsförslag baserade på rådande väderförhållanden.

För att uppnå detta syfte har ett antal mål formulerats. Webbapplikationen ska möjliggöra sökning efter en ort och hämta geografisk information i form av latitud och longitud via ett geokodnings-API. Dessa koordinater ska därefter användas för att hämta väderdata från ett väder-API, vilket innebär att data från flera webbtjänster integreras i en så kallad mashup. Informationen ska presenteras i ett dynamiskt användargränssnitt där innehållet uppdateras med hjälp av JavaScript, vilket ger applikationen ett mer interaktivt och applikationsliknande beteende.

Vidare ska webbapplikationen utformas med responsiv design så att den fungerar på olika typer av enheter. Styling ska implementeras med hjälp av SCSS för att skapa en strukturerad och underhållbar kodbas, och animationer ska användas för att förbättra användarupplevelsen och tydliggöra interaktioner i gränssnittet.

Projektet ska även använda en automatiserad arbetsprocess för att sammanställa och optimera webbapplikationen samt möjliggöra automatisk publicering. Versionshantering ska ske med Git, vilket gör det möjligt att strukturera och dokumentera utvecklingsarbetet.

Målet med projektet är därmed att skapa en fullt fungerande webbapplikation som uppfyller kursens krav och samtidigt demonstrerar hur flera webbtjänster kan integreras i en sammanhängande lösning.

5.4 Författarens bidrag

Arbetet i detta projekt har genomförts individuellt av författaren. Samtliga delar, inklusive planering, design, implementation, testning samt rapportskrivning, har utförts av författaren.

Externa resurser har använts i form av dokumentation för de API:er och verktyg som använts, men ingen direkt hjälp från andra personer har bidragit till projektets genomförande.

6 Teori

För att förstå hur webbapplikationen i detta projekt är uppbyggd krävs grundläggande kunskap om ett antal centrala begrepp och tekniker inom modern webbutveckling.

En viktig del av projektet är användningen av API:er (Application Programming Interfaces), vilket är gränssnitt som möjliggör kommunikation mellan olika system [1]. I detta projekt används externa webbtjänster för att hämta data i realtid. Dessa API:er levererar data i formatet JSON (JavaScript Object Notation), vilket är ett lättviktigt och strukturerat dataformat som är vanligt förekommande vid kommunikation mellan klient och server [2].

Webbapplikationen bygger på principen om en så kallad mashup, vilket innebär att data från flera olika källor kombineras till en gemensam applikation [3]. I detta fall används ett geokodnings-API för att omvandla en ort till geografiska koordinater, vilka sedan används för att hämta väderdata från ett väder-API. På så sätt samverkar flera webbtjänster för att skapa en mer komplett lösning.

För att hämta data från dessa webbtjänster används tekniken AJAX (Asynchronous JavaScript and XML), vilket gör det möjligt att skicka och ta emot data utan att ladda om webbsidan [4]. I detta projekt används den moderna JavaScript-funktionen fetch() för att utföra dessa anrop. Resultatet av API-anropen används sedan för att dynamiskt uppdatera webbapplikationens innehåll genom manipulation av DOM (Document Object Model), vilket är en representation av webbsidans struktur [5].

För styling av webbapplikationen används SCSS, vilket är en vidareutveckling av CSS som möjliggör användning av variabler, nesting och modulär kodstruktur [6]. Detta underlättar organisering och underhåll av stilmallar, särskilt i större projekt.

Webbapplikationen är även byggd med responsiv design, vilket innebär att layout och innehåll anpassas efter olika skärmstorlekar och enheter [7]. Detta är en viktig aspekt i modern webbutveckling då användare ofta besöker webbplatser från både datorer och mobila enheter.

För att effektivisera utvecklingsprocessen används en automatiserad arbetsprocess med hjälp av byggverktyget Vite, vilket hanterar utvecklingsserver, bundling och optimering av webbapplikationen [8].

Versionshantering i projektet sker med hjälp av Git, vilket gör det möjligt att spåra förändringar i koden över tid [9]. Git används tillsammans med plattformen GitHub, där källkoden lagras och hanteras. För att automatisera publiceringen används GitHub Actions, vilket möjliggör att applikationen byggs och publiceras automatiskt vid ändringar i koden [10].

Slutligen används även animationer i webbapplikationen, vilket är visuella effekter som skapas med CSS för att förbättra användarupplevelsen och ge återkoppling vid interaktion [11].

7 Metod

Projektet har genomförts med en iterativ utvecklingsmetod, där funktionalitet har implementerats och testats stegvis. Detta har möjliggjort en successiv uppbyggnad av webbapplikationen samt kontinuerlig verifiering av att de uppsatta målen uppfylls.

För att hämta data från externa webbtjänster har JavaScript och asynkrona anrop använts. Specifikt har fetch() använts för att utföra HTTP-förfrågningar mot API:er och hämta data i JSON-format [2,12]. Först används ett geokodnings-API för att omvandla en användares sökning till geografiska koordinater. Dessa koordinater används därefter i ett väder-API för att hämta aktuell väderinformation och prognos. Denna metod möjliggör integration av flera datakällor i en sammanhängande lösning, vilket är centralt för mashup-konceptet [3].

Användargränssnittet uppdateras dynamiskt genom manipulation av DOM med hjälp av JavaScript [5]. Detta innebär att innehållet kan ändras utan att sidan behöver laddas om, vilket bidrar till ett mer interaktivt och applikationsliknande beteende.

För att strukturera och organisera koden har en modulär uppdelning använts, där funktionalitet delas upp i separata filer för API-anrop, användargränssnitt och hjälpfunktioner. Detta underlättar både utveckling och underhåll av applikationen.

Styling av webbapplikationen har implementerats med SCSS, vilket möjliggör användning av variabler och uppdelning av stilmallar i mindre delar [6]. Detta bidrar till en mer strukturerad och återanvändbar kodbas.

Webbapplikationen har utvecklats med responsiv design genom användning av moderna layouttekniker såsom CSS Grid och media queries, vilket gör att layouten anpassas till olika skärmstorlekar [7,13].

För att förbättra användarupplevelsen har animationer implementerats med CSS. Dessa används bland annat för att visa en loader vid datahämtning samt för att skapa mjuka övergångar i gränssnittet [11].

En automatiserad arbetsprocess har använts med hjälp av byggverktyget Vite, vilket hanterar utvecklingsserver, bundling av filer och optimering av webbapplikationen inför publicering [8].

Versionshantering har skett med Git, vilket möjliggör spårning av förändringar i koden över tid [9]. Projektet har lagrats på GitHub, där även automatisk publicering har konfigurerats med GitHub Actions [10]. Denna metod innebär att webbapplikationen byggs och publiceras automatiskt vid uppdateringar i källkoden.

Genom att kombinera dessa metoder har en strukturerad utvecklingsprocess möjliggjorts, där både funktionella och tekniska krav kunnat uppfyllas.

8 Konstruktion

Webbapplikationen är uppbyggd med en modulär struktur där olika delar av funktionaliteten är uppdelade i separata filer. Detta inkluderar moduler för API-anrop, användargränssnitt, lagring av data samt hjälpfunktioner. Denna uppdelning gör koden mer lättläst och underhållbar.

Applikationens huvudlogik hanteras i en central fil där användarinteraktion, API-anrop och rendering av innehåll kopplas samman. När användaren gör en sökning initieras ett flöde där data hämtas från externa webbtjänster och därefter presenteras i gränssnittet.

8.1 Hantering av användarinput

Användarens interaktion med applikationen sker via ett sökformulär där en ort anges. När formuläret skickas in fångas händelsen upp i JavaScript och standardbeteendet förhindras, vilket gör att sidan inte laddas om.

Kodexempel 1:

```
form.addEventListener("submit", handleSubmit);

async function handleSubmit(event) {
  event.preventDefault();
  await runSearch(cityInput.value.trim());
}
```

Som kodexempel 1 visar gör denna metod det möjligt att hantera användarinput dynamiskt och starta applikationens huvudflöde.

8.2 Integration av API:er

Webbapplikationen använder två externa API:er. Först används ett geokodnings-API för att omvandla användarens sökning till geografiska koordinater. Därefter används dessa koordinater för att hämta väderdata från ett väder-API.

Kodexempel 2 (geokodning):

```
export async function getLocationByName(query) {

  const response = await fetch(url);

  const data = await response.json();

  return data.results[0];

}
```

Kodexempel 3 (väderdata):

```
export async function getWeatherByCoordinates(latitude, longitude) {

  const response = await fetch(url);

  return await response.json();

}
```

Genom att använda resultatet från det första API-anropet i det andra skapas en koppling mellan tjänsterna, vilket är centralt för mashup-konceptet.

8.3 Dynamisk rendering av innehåll

Efter att data har hämtats från API:erna uppdateras användargränssnittet dynamiskt genom manipulation av DOM. Detta innebär att HTML-innehåll genereras och sätts in i dokumentet med hjälp av JavaScript.

Kodexempel 4:

```
activityListEl.innerHTML = activities

  .map((activity) => `<li>${activity}</li>`)

  .join("");
```

Denna metod används för att presentera både väderinformation och aktivitetsförslag, vilket ger en mer interaktiv användarupplevelse.

8.4 Kartintegration

För att visualisera den valda platsen används biblioteket Leaflet. Kartan initieras en gång och uppdateras sedan med nya koordinater vid varje sökning.

Kodexempel 5:

```
map.setView([latitude,longitude],11);

marker=L.marker([latitude,longitude])
  .addTo(mapp)
  .bindPopup(label)
  .openPopup();
```

Detta gör det möjligt att koppla samman geografiska data med en visuell representation.

8.5 Lagring av tidigare sökningar

För att förbättra användarupplevelsen lagras tidigare sökningar i webbläsarens localStorage. Detta gör det möjligt att återanvända tidigare sökningar utan att behöva skriva in dem igen.

Kodexempel 6:

```
localStorage.setItem(STORAGE_KEY, JSON.stringify(updated));
```

Vid uppstart av applikationen hämtas dessa data och presenteras som klickbara alternativ.

8.6 Styling och layout

Webbapplikationens utseende har implementerats med SCSS, där stilmallar är uppdelade i flera filer baserat på funktion, exempelvis komponenter och layout. Variabler används för färger och storlekar, vilket gör det enkelt att upprätthålla en enhetlig grafisk profil.

Layouten är uppbyggd med moderna CSS-tekniker såsom Grid och Flexbox, vilket möjliggör en responsiv design där innehållet anpassas efter olika skärmstorlekar.

8.7 Animationer

Animationer har implementerats med CSS för att förbättra användarupplevelsen. En loader används vid datahämtning för att ge visuell feedback till användaren. Vidare används en fade-in-animation när nytt innehåll visas, vilket skapar en mjuk övergång i gränssnittet.

Mindre animationer används även vid interaktion, exempelvis hover-effekter på knappar, för att tydliggöra klickbara element.

8.8 Automatiska arbetsprocess och publicering

För att effektivisera utvecklingsprocessen används byggverktyget Vite, vilket hanterar utvecklingsserver och optimering av projektet inför publicering.

Versionshantering sker med Git och projektet lagras på GitHub. Automatisk publicering har implementerats med GitHub Actions, där en workflow-fil används för att bygga och publicera webbapplikationen till GitHub Pages vid varje uppdatering av koden.

9 Resultat

Resultatet av projektet är en fungerande webbapplikation som uppfyller de krav och mål som definierats tidigare i rapporten. Applikationen gör det möjligt för användaren att söka efter en ort och få tillgång till aktuell väderinformation, en korttidsprognos samt aktivitetsförslag baserade på rådande väderförhållanden.

Webbapplikationen hämtar geografisk information via ett geokodnings-API, vilket används för att erhålla latitud och longitud för den valda platsen. Dessa koordinater används därefter för att hämta väderdata från ett väder-API. Resultatet är att data från flera webbtjänster kombineras i en sammanhängande lösning, vilket uppfyller kravet på en mashup.

Användargränssnittet uppdateras dynamiskt med hjälp av JavaScript, vilket innebär att innehållet förändras utan att sidan behöver laddas om. Detta ger applikationen ett applikationsliknande beteende.

Applikationen innehåller även en kartfunktion där den valda platsen visualiseras, samt en funktion för att spara och återanvända tidigare sökningar. Dessa funktioner bidrar till att förbättra användarupplevelsen.

Webbapplikationen är responsiv och anpassar sig till olika skärmstorlekar, vilket gör att den fungerar på både datorer och mobila enheter. Styling har implementerats med SCSS och ger applikationen en enhetlig grafisk profil.Animationer används i gränssnittet, bland annat i form av en loader vid datahämtning samt visuella övergångar vid rendering av innehåll.

En automatiserad arbetsprocess har implementerats med hjälp av Vite, vilket möjliggör effektiv utveckling och optimering av webbapplikationen. Versionshantering har skett med Git och projektet har publicerats med hjälp av GitHub Actions, vilket innebär att applikationen byggs och publiceras automatiskt vid uppdateringar i koden.

Sammanfattningsvis visar resultatet att samtliga centrala funktioner har implementerats och att webbapplikationen fungerar enligt de mål som sattes upp i projektets inledande fas.

10 Slutsatser

Syftet med projektet var att utveckla en webbapplikation som kombinerar data från flera webbtjänster i en sammanhängande lösning. Resultatet visar att detta syfte har uppnåtts, då applikationen framgångsrikt integrerar data från ett geokodnings-API och ett väder-API i en fungerande mashup.

En styrka i projektet är den tydliga kopplingen mellan de olika tekniker som använts, där API-integration, dynamisk rendering med JavaScript och modulär kodstruktur tillsammans skapar en applikation med ett applikationsliknande beteende. Även användningen av SCSS och responsiv design bidrar till en enhetlig och användarvänlig lösning.

En annan positiv aspekt är den automatiserade arbetsprocessen, där Vite och GitHub Actions används för att effektivisera utveckling och publicering. Detta visar på en förståelse för moderna arbetsflöden inom webbutveckling.

Samtidigt finns det områden där lösningen kan förbättras. Exempelvis skulle fler funktioner kunna implementeras, såsom integration av ytterligare API:er för att visa sevärdheter eller bilder kopplade till den valda platsen. Även användargränssnittet skulle kunna vidareutvecklas med mer avancerad interaktivitet och fler anpassningsmöjligheter.

En annan möjlig förbättring är att implementera mer avancerad felhantering samt bättre hantering av olika typer av användarinput. Detta skulle öka applikationens robusthet.

Sammanfattningsvis visar projektet att det är möjligt att kombinera flera webbtjänster till en fungerande webbapplikation med hjälp av moderna webbutvecklingstekniker. Projektet har bidragit till ökad förståelse för API-integration, automatiserade arbetsprocesser och utveckling av dynamiska webbapplikationer.

Källförteckning

[1] IBM. What is an API? [Internet]. Tillgänglig från: https://www.ibm.com/topics/api

[2] MDN Web Docs. JSON [Internet]. Tillgänglig från: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON

[3] Wikipedia. Mashup (web application hybrid) [Internet]. Tillgänglig från: https://en.wikipedia.org/wiki/Mashup_(web_application_hybrid)

[4] MDN Web Docs. AJAX [Internet]. Tillgänglig från: https://developer.mozilla.org/en-US/docs/Web/Guide/AJAX

[5] MDN Web Docs. Document Object Model (DOM) [Internet]. Tillgänglig från: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

[6] Sass. Sass Basics [Internet]. Tillgänglig från: https://sass-lang.com/guide

[7] MDN Web Docs. Responsive design [Internet]. Tillgänglig från: https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Responsive_Design

[8] Vite. Guide [Internet]. Tillgänglig från: https://vitejs.dev/guide/

[9] Git Documentation. About Git [Internet]. Tillgänglig från: https://git-scm.com/docs

[10] GitHub Docs. GitHub Actions [Internet]. Tillgänglig från: https://docs.github.com/en/actions

[11] MDN Web Docs. CSS Animations [Internet]. Tillgänglig från: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Animations

[12] MDN Web Docs. Fetch API [Internet]. Tillgängliga från: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[13] MDN Web Docs. CSS Grid Layout [Internet]. Tillgänglig från: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout